



Matplotlib for Data Science

Day 5 — Lecture Notes & Practical Tasks

Pyplot · Line/Scatter/Bar/Pie · Customizing · Subplots

1. Introduction to Matplotlib

Matplotlib is the fundamental data visualization library in Python. It provides both a quick Pyplot interface and a robust Object-Oriented interface.

1.1 Line and Scatter Plots

```
import matplotlib.pyplot as plt
import numpy as np

# Data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Line Plot
plt.plot(x, y1, color='blue', linestyle='-', linewidth=2, label='Sine')

# Scatter Plot
# Note: typically used for comparing two different continuous variables
x_scatter = np.random.rand(50) * 10
y_scatter = np.random.rand(50) * 10
plt.scatter(x_scatter, y_scatter, color='red', marker='x', s=50,
            label='Random Points')

plt.legend()
plt.show()
```

1.2 Tasks — Line & Scatter

Task 1: Create a line plot showing the squares of numbers from 1 to 10. Make the line dashed and green.

Task 2: Plot two lines on the same graph: $y=x^2$ and $y=x^3$. Add a legend to distinguish them.

Task 3: Create a scatter plot of height vs weight using dummy data. Color the points purple and use triangles as markers.

2. Categorical & Distribution Plots

Bar charts compare categorical data. Histograms show the distribution of a single numerical variable. Pie charts show proportions.

2.1 Bar, Histogram, and Pie

```
import matplotlib.pyplot as plt
import numpy as np

# Bar Chart
categories = ['Apples', 'Bananas', 'Cherries']
counts = [40, 25, 15]
plt.bar(categories, counts, color=['red', 'yellow', 'darkred'])
plt.title("Fruit Inventory")
plt.show()

# Histogram
data = np.random.randn(1000) # Normal distribution
plt.hist(data, bins=30, edgecolor='black', color='skyblue')
plt.title("Data Distribution")
plt.show()

# Pie Chart
plt.pie(counts, labels=categories, autopct='%1.1f%%', explode=[0.1, 0, 0])
plt.title("Fruit Share")
plt.show()
```

2.2 Tasks — Categorical

Task 1: Create a horizontal bar chart (plt.barh) showing population of 5 different countries.

Task 2: Generate 500 random ages between 0 and 80. Plot a histogram with 10 bins. Add a black edge color to the bins.

Task 3: Create a pie chart showing your daily time allocation (sleep, work, study, leisure). Explode the 'study' slice.

3. Customizing & Subplots

A good plot requires clear labels, titles, grids, and proper axis limits. Subplots allow multiple charts in one figure.

3.1 Titles, Labels, Limits, Grid

```
import matplotlib.pyplot as plt

plt.plot([1,2,3], [10,20,15], label='Sales')
plt.title("Q1 Sales Report", fontsize=16, fontweight='bold')
plt.xlabel("Month", fontsize=12)
plt.ylabel("Revenue ($)", fontsize=12)

# Axis limits
plt.xlim(0, 4)
plt.ylim(0, 25)

# Custom ticks
plt.xticks([1,2,3], ['Jan', 'Feb', 'Mar'])

plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
```

3.2 Subplots

```
import matplotlib.pyplot as plt

# 1 row, 2 columns subplot grid
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))

# First plot
axes[0].plot([1,2,3], [3,2,1], color='blue')
axes[0].set_title('Descending')

# Second plot
axes[1].plot([1,2,3], [1,2,3], color='red')
axes[1].set_title('Ascending')

plt.tight_layout() # Fixes spacing
plt.savefig('my_subplots.png', dpi=300) # Save figure
plt.show()
```

3.3 Tasks — Customization & Subplots

Task 1: Take your line plot from Task 1.1 and add a title, x/y labels, a grid, and set x-axis limits from 0 to 12.

Task 2: Create a 2x2 subplot grid. Plot a line, scatter, bar, and histogram in the 4 axes respectively. Add titles to each.

Task 3: Save the 2x2 grid as a high-resolution JPG image using `plt.savefig()`.